Homework 4 Solution

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm
DATA = "hwk4_data/"
plt.rcParams.update({"figure.dpi": 100, "font.size": 11})
np.set_printoptions(precision=5, suppress=True)
ALPHAS = [0.1, 0.01, 0.001, 0.0001]
STYLES = {0.1: ("-", "o"), 0.01: ("--", "s"), 0.001: ("-.", "^"), 0.0001: (":", "d")}
```

Problem 1: Gradient descent on $f(x,y) = (a-x)^2 + b(y-x^2)^2$

With $a = 1$, $b = 100$ this is the Rosenbrock function. Its unique global minimum is $f(1,1) = 0$, located at the bottom of a narrow, curved, parabolic valley $y = x^2$.

The gradient is

$$\nabla f = \begin{pmatrix} -2(a-x) - 4bx(y-x^2) \\ 2b(y-x^2) \end{pmatrix},$$

and gradient descent iterates $\mathbf{z}_{k+1} = \mathbf{z}_k - \alpha \nabla f(\mathbf{z}_k)$ from the start point $(-1,1)$. Each run uses a fixed budget of 100000 iterations, and stops early (recording the failure) if the iterate overflows.

```
a, b = 1.0, 100.0
def f(x, y): return (a-x)**2 + b*(y-x**2)**2
def grad(z):
    x, y = z
    return np.array([-2*(a-x) - 4*b*x*(y-x**2), 2*b*(y-x**2)])
zstar = np.array([1.0, 1.0])

def gd(alpha, z0=(-1.0, 1.0), iters=100000, blow=1e8):
    z = np.array(z0, float)
    path = [z.copy()]; err = [np.linalg.norm(z-zstar)]
    for k in range(iters):
        z = z - alpha*grad(z)
        if not np.all(np.isfinite(z)) or np.linalg.norm(z) > blow:
            err.append(np.inf); path.append(np.full(2, np.nan)); break
        path.append(z.copy()); err.append(np.linalg.norm(z-zstar))
    return np.array(path), np.array(err)

res = {al: gd(al) for al in ALPHAS}
for al, (p, e) in res.items():
    status = "DIVERGED after %d iterations" % (len(e)-1) if not np.isfinite(e[-1]) else
"finite"
    last = p[-1] if np.isfinite(e[-1]) else p[-2]
    print(f"alpha={al:<7} {status:32s} last finite point={last}  final distance to (1,1)=
{e[-1]:.3e}")
```

```
alpha=0.1     DIVERGED after 4 iterations      last finite point=[160397.53376   5116.392
]  final distance to (1,1)=inf
alpha=0.01    DIVERGED after 6 iterations      last finite point=[8087.8772   322.12549]
final distance to (1,1)=inf
alpha=0.001   finite                           last finite point=[1. 1.]  final distance to
(1,1)=1.613e-13
alpha=0.0001  finite                           last finite point=[0.99247 0.98497]  final
distance to (1,1)=1.680e-02
```

Stability analysis

Near the minimiser, gradient descent on a quadratic with Hessian H is stable iff $\alpha < 2/\lambda_{\max}(H)$. We compute the Hessian

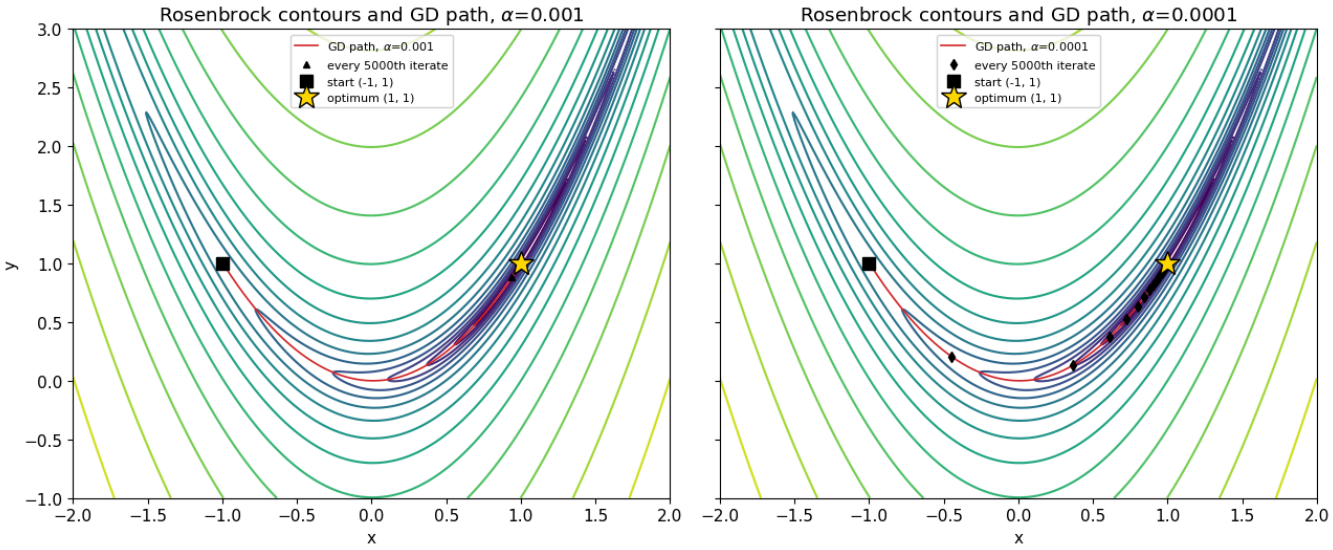
$$H = \begin{pmatrix} 2 - 4b\left(y - x^2\right) + 8bx^2 & -4bx \\ -4bx & 2b \end{pmatrix}.$$

```
def hess(x, y):
    return np.array([[2 - 4*b*(y-x**2) + 8*b*x**2, -4*b*x], [-4*b*x, 2*b]])
H = hess(1, 1); ev = np.linalg.eigvalsh(H)
print("Hessian at (1,1):\n", H)
print("eigenvalues:", ev, " condition number:", ev[-1]/ev[0])
print("stability limit 2/lambda_max =", 2/ev[-1])
print("Hessian lambda_max at start (-1,1):", np.linalg.eigvalsh(hess(-1,1))[-1], "-> local
limit", 2/np.linalg.eigvalsh(hess(-1,1))[-1])
print("asymptotic contraction factor per step 1 - alpha*lambda_min:", {al: 1-al*ev[0] for
al in ALPHAS})
```

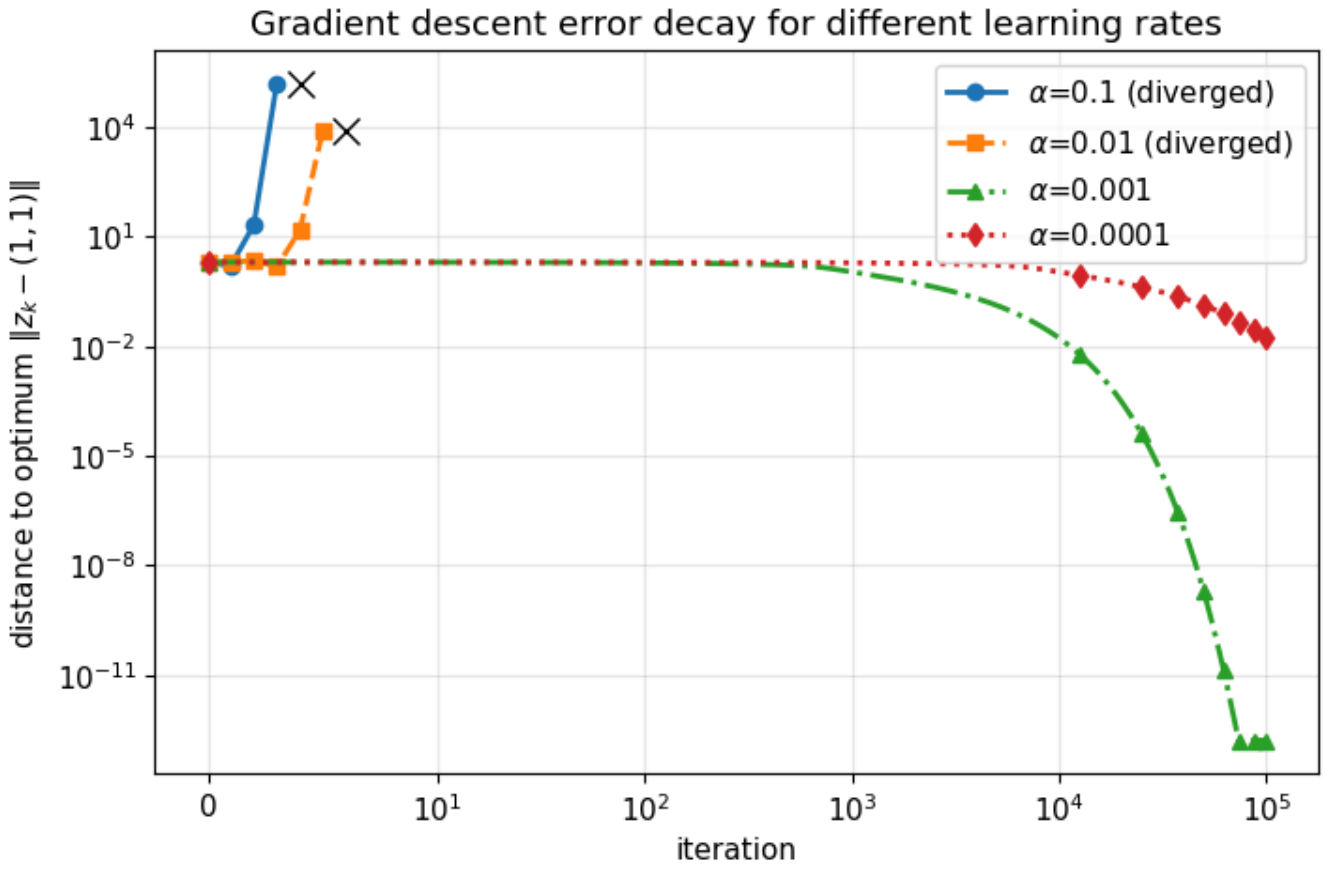
```
Hessian at (1,1):
 [[ 802. -400.]
 [-400.  200.]]
eigenvalues: [   0.39936 1001.60064]  condition number: 2508.009601277454
stability limit 2/lambda_max = 0.0019968038374381654
Hessian lambda_max at start (-1,1): 1001.6006392325123 -> local limit 0.0019968038374381654
asymptotic contraction factor per step 1 - alpha*lambda_min: {0.1: 0.9600639232512369,
0.01: 0.9960063923251237, 0.001: 0.9996006392325124, 0.0001: 0.9999600639232512}
```

The largest Hessian eigenvalue at the minimum is about 1001.6, so the stable step size is $\alpha < 2/1001.6 \approx 0.002$. Hence $\alpha = 0.1$ and $\alpha = 0.01$ are far above the limit and diverge (the steep direction is amplified by $|1 - \alpha\lambda_{\max}| > 1$ each step, so the iterates explode to overflow), while $\alpha = 0.001$ and 0.0001 are stable. The smallest eigenvalue is only about 0.4, giving a condition number of about 2500; progress along the valley floor contracts by only $1 - \alpha\lambda_{\min}$ per step, which is 0.9996 for $\alpha = 0.001$ and 0.99996 for $\alpha = 0.0001$.

```
xs = np.linspace(-2, 2, 400); ys = np.linspace(-1, 3, 400)
XX, YY = np.meshgrid(xs, ys)
fig, axs = plt.subplots(1, 2, figsize=(13, 5.5), sharey=True)
for ax, al in zip(axs, [0.001, 0.0001]):
    cs = ax.contour(XX, YY, f(XX, YY), levels=np.logspace(-1, 3.5, 16), norm=LogNorm(),
cmap="viridis")
    p = res[al][0]
    ls, mk = STYLES[al]
    ax.plot(p[:,0], p[:,1], color="tab:red", ls="-", lw=1.2, label=f"GD path, $\\alpha$=
{al}")
    ax.plot(p[::5000,0], p[::5000,1], "k"+mk, ms=5, ls="none", label="every 5000th
iterate")
    ax.plot(*p[0], "ks", ms=9, label="start (-1, 1)")
    ax.plot(*zstar, "*", color="gold", mec="k", ms=18, label="optimum (1, 1)")
    ax.set_xlabel("x"); ax.set_title(f"Rosenbrock contours and GD path, $\\alpha$={al}")
    ax.legend(loc="upper center", fontsize=8)
axs[0].set_ylabel("y"); plt.tight_layout(); plt.show()
```




```
fig, ax = plt.subplots(figsize=(8, 5))
for al in ALPHAS:
    e = res[al][1]; ls, mk = STYLES[al]
    k = np.arange(len(e))
    ax.semilogy(k, np.where(np.isfinite(e), e, np.nan), ls=ls, lw=2, marker=mk,
markevery=max(1, len(e)//8), ms=6,
                label=f"$\\alpha$={al}" + (" (diverged)" if not np.isfinite(e[-1]) else
""))
    if not np.isfinite(e[-1]):
        ax.plot(len(e)-1, np.nanmax(np.where(np.isfinite(e), e, np.nan)), "kx", ms=10)
ax.set_xscale("symlog", linthresh=10)
ax.set_xlabel("iteration"); ax.set_ylabel(r"distance to optimum $\|z_k-(1,1)\|$")
ax.set_title("Gradient descent error decay for different learning rates")
ax.legend(); ax.grid(alpha=.3, which="both"); plt.show()
for al in ALPHAS:
    e = res[al][1]; print(f"alpha={al}: min error reached = {np.nanmin(e):.3e}, final =
{e[-1]:.3e}")
```




```
alpha=0.1: min error reached = 1.600e+00, final = inf
alpha=0.01: min error reached = 1.517e+00, final = inf
alpha=0.001: min error reached = 1.613e-13, final = 1.613e-13
alpha=0.0001: min error reached = 1.680e-02, final = 1.680e-02
```

Findings on convergence

- $\alpha = 0.1, 0.01$ **(too large):** exceed the stability limit $2/L \approx 0.002$, so gradient descent diverges.
- $\alpha = 0.001$ **(just inside the limit):** converges to $(1, 1)$ in the fixed budget. It is the best choice here, though only about a factor of two below the stability limit, so it is sensitive to the start point (a start with a larger local curvature could make it diverge).
- $\alpha = 0.0001$ **(small):** stable but slow. The valley is narrow and curved with a condition number of about 2500, so the step size is dictated by the steep direction while progress along the flat valley floor is governed by the tiny eigenvalue ≈ 0.4; it needs about ten times more iterations than $\alpha = 0.001$.
- General lesson: for ill-conditioned problems the step size must be $< 2/\lambda_{\max}$, and the convergence rate is limited by the condition number $\lambda_{\max}/\lambda_{\min}$, motivating momentum, adaptive step sizes or second-order methods.

Problem 2: Linear regression with SGD

We fit $\widehat{y} = wx + b$. First inspect the data.

```
X1 = np.load(DATA+"inputdata.npy").ravel(); y = np.load(DATA+"target.npy").ravel()
n = len(y)
print("shapes:", X1.shape, y.shape, " x in [%.3f, %.3f], y in [%.3f, %.3f]" % (X1.min(),
X1.max(), y.min(), y.max()))
Xd = np.column_stack([np.ones(n), X1])        # columns: bias, x
w_ne = np.linalg.solve(Xd.T@Xd, Xd.T@y)       # normal equations
loss = lambda w, lam=0.0: 0.5*np.mean((Xd@w - y)**2) + 0.5*lam*w[1]**2
print("normal equation: b = %.4f, w = %.4f, loss = %.4f" % (w_ne[0], w_ne[1], loss(w_ne)))
```

```
shapes: (100,) (100,)  x in [0.011, 1.974], y in [3.238, 11.044]
normal equation: b = 4.2151, w = 2.7701, loss = 0.4033
```

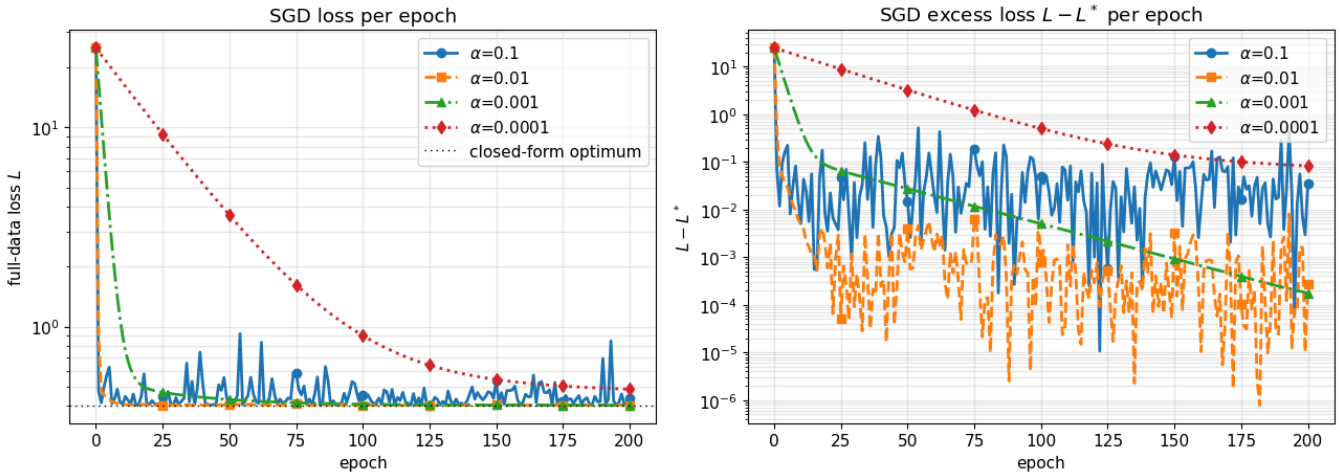
The data are one-dimensional ($N = 100$ points, $x \in [0, 2]$) with an approximately linear target. We use the loss $L(w, b) = \frac{1}{2N} \sum_i \left(w x_i + b - y_i \right)^2$, whose minimiser is given in closed form by the normal equations $\left(X^T X \right) \theta = X^T y$ with $\theta = (b, w)$ (bias column included in X). SGD uses one sample per update, $\theta \leftarrow \theta - \alpha \left(\widehat{y}_i - y_i \right) \mathbf{x}_i$, with a fresh random shuffle every epoch (fixed seed). The curves below record the full-data loss after each epoch.

```
def sgd(alpha, lam=0.0, epochs=200, seed=0):
    rng = np.random.default_rng(seed)
    th = np.zeros(2); hist = [loss(th, lam)]
    for ep in range(epochs):
        for i in rng.permutation(n):
            g = (Xd[i]@th - y[i])*Xd[i]
            g[1] += lam*th[1]              # L2 penalty on the slope only (bias not
penalised)
            th = th - alpha*g
        hist.append(loss(th, lam))
    return th, np.array(hist)

sg = {al: sgd(al) for al in ALPHAS}
Lstar = loss(w_ne)
print("closed-form: b=%.4f w=%.4f loss=%.5f" % (w_ne[0], w_ne[1], Lstar))
for al, (th, h) in sg.items():
    print(f"alpha={al:<7} b={th[0]:.4f} w={th[1]:.4f} final loss={h[-1]:.5f}  excess=
{h[-1]-Lstar:.2e}")
```

```
closed-form: b=4.2151 w=2.7701 loss=0.40329
alpha=0.1     b=4.1810 w=3.0332 final loss=0.43816  excess=3.49e-02
alpha=0.01    b=4.2240 w=2.7841 final loss=0.40357  excess=2.76e-04
alpha=0.001   b=4.1828 w=2.8010 final loss=0.40346  excess=1.73e-04
alpha=0.0001  b=3.4553 w=3.3367 final loss=0.48532  excess=8.20e-02
```

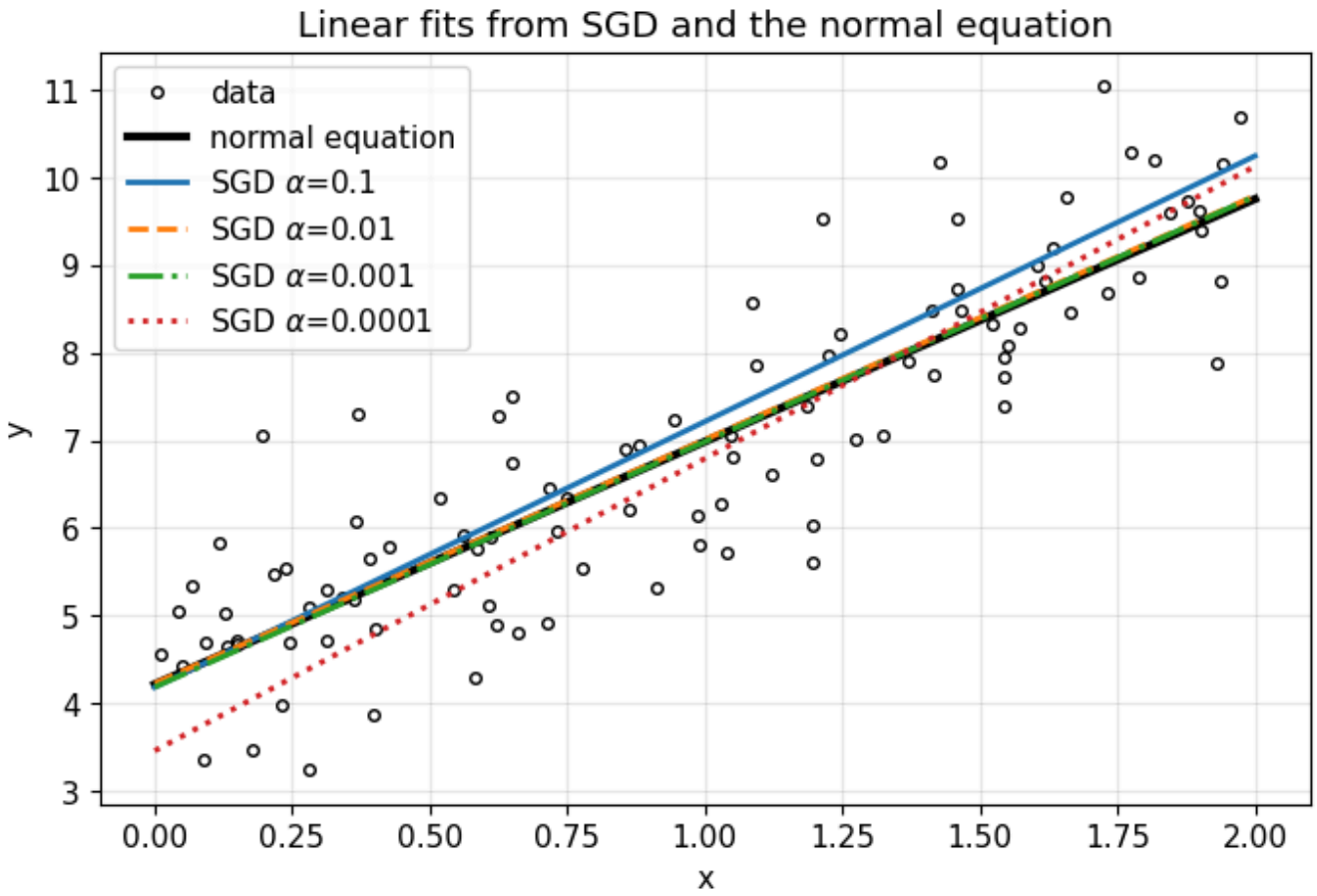
```
fig, axs = plt.subplots(1, 2, figsize=(13, 4.8))
for al in ALPHAS:
    h = sg[al][1]; ls, mk = STYLES[al]
    axs[0].semilogy(h, ls=ls, lw=2, marker=mk, markevery=25, ms=6, label=f"$\\alpha$={al}")
    axs[1].semilogy(np.maximum(h-Lstar, 1e-16), ls=ls, lw=2, marker=mk, markevery=25, ms=6,
label=f"$\\alpha$={al}")
axs[0].axhline(Lstar, color="k", lw=1, ls=(0,(1,3)), label="closed-form optimum")
axs[0].set_title("SGD loss per epoch"); axs[0].set_ylabel("full-data loss $L$")
axs[1].set_title("SGD excess loss $L-L^*$ per epoch"); axs[1].set_ylabel("$L-L^*$")
for ax in axs:
    ax.set_xlabel("epoch"); ax.legend(); ax.grid(alpha=.3, which="both")
plt.tight_layout(); plt.show()
```



What we find about the learning rate

- $\alpha = 0.1$ converges in the fewest epochs but, because the gradient estimate is noisy (one sample), the iterate keeps fluctuating around the optimum; the noise floor of the excess loss grows with α (3.5×10^{-2} for $\alpha = 0.1$, versus $\sim 10^{-4}$ for 0.01 and 0.001).
- Smaller α gives a smoother path but is slower; $\alpha = 0.0001$ has clearly not converged after 200 epochs (20000 updates), with excess loss 8×10^{-2}. Here $\alpha = 0.01$ is the best trade-off.
- Since each per-sample Hessian $\mathbf{x}_i \mathbf{x}_i^T$ has $\lambda_{\max} \le 1 + x_i^2 \le 5$ here, all four rates are below the stability limit (≈ 0.4), so none diverge. In practice a decaying step size removes the noise floor.

```
xx = np.linspace(0, 2, 100); Xp = np.column_stack([np.ones_like(xx), xx])
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(X1, y, "ko", ms=4, mfc="none", label="data")
ax.plot(xx, Xp@w_ne, "k-", lw=3, label="normal equation")
for al in ALPHAS:
    ls, mk = STYLES[al]
    ax.plot(xx, Xp@sg[al][0], ls=ls, lw=2, label=f"SGD $\\alpha$={al}")
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_title("Linear fits from SGD and the normal
equation")
ax.legend(); ax.grid(alpha=.3); plt.show()
```



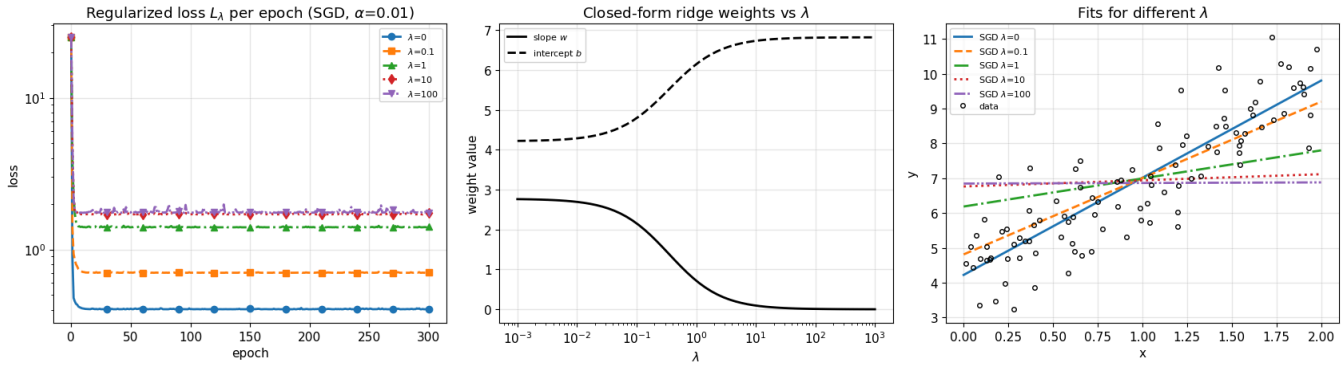
Adding L2 regularization (ridge regression)

The regularized loss is $L_\lambda = \frac{1}{2N}\sum_i \left(wx_i + b - y_i\right)^2 + \frac{\lambda}{2}w^2$ (the bias b is not penalised). Its SGD per-sample gradient adds λw, and its closed-form minimiser is $\theta = \left(X^TX/N + \lambda D\right)^{-1}X^Ty/N$ with $D = \mathrm{diag}(0, 1)$. We use $\alpha = 0.01$ (a good compromise between speed and noise) with 300 epochs.

```
def ridge_cf(lam):
    return np.linalg.solve(Xd.T@Xd/n + lam*np.diag([0, 1]), Xd.T@y/n)
lams = [0, 0.1, 1, 10, 100]
rows = []
print(f"{'lambda':>7} | {'SGD b':>8} {'SGD w':>8} | {'closed b':>8} {'closed w':>8} |
{'train MSE (closed)':>18}")
sgr = {}
for lam in lams:
    th, h = sgd(0.01, lam, epochs=300); sgr[lam] = (th, h)
    cf = ridge_cf(lam)
    print(f"{lam:7g} | {th[0]:8.4f} {th[1]:8.4f} | {cf[0]:8.4f} {cf[1]:8.4f} |
{np.mean((Xd@cf-y)**2):18.4f}")
```

```
 lambda |    SGD b    SGD w | closed b closed w | train MSE (closed)
      0 |   4.2239   2.7894 |   4.2151   2.7701 |             0.8066
    0.1 |   4.8130   2.1926 |   4.7934   2.1552 |             0.9391
      1 |   6.1879   0.8060 |   6.1440   0.7189 |             2.2812
     10 |   6.7641   0.1757 |   6.7318   0.0938 |             3.3168
    100 |   6.8469   0.0169 |   6.8109   0.0097 |             3.4771
```

```
fig, axs = plt.subplots(1, 3, figsize=(17, 4.8))
lstyles = ["-", "--", "-.", ":", (0,(5,1,1,1,1,1))]; marks = ["o", "s", "^", "d", "v"]
for lam, ls, mk in zip(lams, lstyles, marks):
    th, h = sgr[lam]
    axs[0].semilogy(h, ls=ls, lw=2, marker=mk, markevery=30, ms=6, label=f"$\\lambda$=
{lam:g}")
    axs[2].plot(xx, Xp@th, ls=ls, lw=2, label=f"SGD $\\lambda$={lam:g}")
axs[0].set_title(r"Regularized loss $L_\lambda$ per epoch (SGD, $\alpha$=0.01)");
axs[0].set_xlabel("epoch"); axs[0].set_ylabel("loss")
ll = np.logspace(-3, 3, 60); W = np.array([ridge_cf(l) for l in ll])
axs[1].semilogx(ll, W[:,1], "k-", lw=2, label="slope $w$"); axs[1].semilogx(ll, W[:,0], "k-
-", lw=2, label="intercept $b$")
axs[1].set_title("Closed-form ridge weights vs $\\lambda$");
axs[1].set_xlabel("$\\lambda$"); axs[1].set_ylabel("weight value")
axs[2].plot(X1, y, "ko", ms=4, mfc="none", label="data"); axs[2].set_title("Fits for
different $\\lambda$")
axs[2].set_xlabel("x"); axs[2].set_ylabel("y")
for ax in axs: ax.legend(fontsize=8); ax.grid(alpha=.3)
plt.tight_layout(); plt.show()
```



Findings on regularization

The SGD ridge solutions agree with the closed-form ridge solutions up to SGD noise and the finite number of epochs (table above; e.g. at $\lambda = 1$ the slope is 0.81 (SGD) versus 0.72 (closed form)). Increasing λ shrinks the slope towards zero and the intercept towards the mean of y: the slope is 2.77, 2.16, 0.72, 0.09, 0.01 for $\lambda = 0, 0.1, 1, 10, 100$, and the training MSE rises from 0.81 to 3.48 (bias-variance trade-off: lower variance, higher bias). Because the loss is averaged over N samples, even $\lambda = 0.1$ shrinks noticeably. With a model this simple (two parameters, 100 points) there is little overfitting, so regularization only hurts the training fit; its benefit shows for high-dimensional or ill-conditioned problems.